

Replicating Table 1, Panel B from Chernozhukov et al. (2017)

Aaron Chotzen-Jenner

June 2, 2026

1 The Original Paper

Chernozhukov, Chetverikov, Demirer, Duflo, Hansen, and Newey (2017), “Double/Debiased/Neyman Machine Learning of Treatment Effects,” *American Economic Review: Papers & Proceedings*, 107(5): 261–265, introduces the Double/Debiased Machine Learning (Double ML) estimator for causal inference in settings where high-dimensional nuisance parameters must be estimated. The central research question is how to obtain valid, root- N consistent estimates of a treatment effect parameter θ_0 when the confounding functions are estimated using flexible machine learning methods that introduce regularization bias. In simpler terms, the paper explains how to measure the true effect of a treatment when machine learning is used to control for other factors that may influence the outcome.

The authors approach this problem with two main methods. First, the estimating equation is made Neyman-orthogonal, meaning it is locally insensitive to small errors in the nuisance parameter estimates. In other words, the equation is designed in a way that small mistakes in the machine learning predictions have very little impact on the final treatment effect estimate. Second, cross-fitting is used to split the sample into folds, training the nuisance functions (everything else that needs to be estimated besides the main effect) on held-out data to avoid overfitting bias. Together, these two steps remove the regularization bias and therefore not distorted by error from the machine learning models.

2 The Result of Interest

The specific result I replicate is the Mean Average Treatment Effect (ATE) reported in Table 1, Panel B, row “ATE (2 fold),” Ensemble column of the online appendix. This result corresponds to the partially linear model with 2-fold cross-fitting. The paper reports a Mean ATE of \$9,043 with a standard error of 1,432.

This result is the headline finding of the paper: after flexibly controlling for income and other confounding factors, 401(k) eligibility increases net financial assets by roughly \$9,000. The underlying model estimates the relationship between financial assets, 401(k) eligibility, and a set of demographic and economic control variables. Rather than relying on a single machine learning algorithm, the Ensemble method combines predictions from

several approaches, including Lasso, Random Forests, Boosting, and Neural Networks. It assigns more weight to the methods that perform best out-of-sample, as the goal is to obtain a more reliable estimate than any single method alone.

I chose this result to replicate because it is the paper’s primary empirical finding, corresponds to the default specification in the replication code (`Master.R`), and illustrates the central idea of Double ML, which is using machine learning to control for complex confounding while still producing a credible estimate of a causal treatment effect.

3 Data and Methods

The data come from the 1991 Survey of Income and Program Participation (SIPP), the same dataset used in Chernozhukov and Hansen (2004). The sample contains 9,915 observations. The outcome variable `net_tfa` is net total financial assets, measured in dollars. The treatment variable `e401` is a binary indicator for eligibility to enroll in a 401(k) plan. The nine covariates used in the model are age, income, years of education, family size, and binary indicators for married status, two-earner household, defined benefit pension, IRA participation, and home ownership.

The partially linear model estimated here takes the form

$$Y = \theta_0 D + g_0(Z) + \zeta, \quad E[\zeta \mid Z, D] = 0, \quad (1)$$

$$D = m_0(Z) + \nu, \quad E[\nu \mid Z] = 0, \quad (2)$$

where Y is net financial assets, D is 401(k) eligibility, Z is the vector of covariates, θ_0 is the Average Treatment Effect (ATE) of interest, and $g_0(Z)$ and $m_0(Z)$ are unknown nuisance functions estimated via machine learning. Equation (1) is the outcome equation, where θ_0 enters linearly and $g_0(Z)$ captures the flexible effect of controls on assets. Equation (2) is the treatment equation, where $m_0(Z) := E[D \mid Z]$ is the propensity score — the probability of 401(k) eligibility given observables.

The key insight of Double ML is that plugging ML estimates of g_0 and m_0 directly into a naive regression for θ_0 introduces regularization bias. Double ML addresses this problem with two key ideas. First, the estimating equation is made Neyman-orthogonal using an orthogonal score such as

$$\psi(W, \theta, \eta) := (Y - \hat{g}(Z) - \theta(D - \hat{m}(Z)))(D - \hat{m}(Z)), \quad (3)$$

which is locally insensitive to small errors in the nuisance estimates $\hat{g}$ and $\hat{m}$. This amounts to a residuals-on-residuals regression: the outcome residual $Y - \hat{g}(Z)$ is regressed on the treatment residual $D - \hat{m}(Z)$, removing the influence of covariates before estimating the treatment effect. Second, cross-fitting is used to avoid overfitting: the nuisance functions are trained on a held-out fold and applied to the remaining fold, so that the same observations are never used for both nuisance estimation and moment evaluation.

`preprocess.R` loads the raw Stata file `sipp1991.dta` from `input/`, selects the eleven relevant variables, and writes a clean CSV to `temp/clean_data.csv`. `analysis.R` loads that file, sources the original helper functions `ML.Functions.R` and `Moment.Functions.R` from `input/`, and calls `DoubleML()` with the partially linear specification above and 2-fold cross-fitting. To reduce computation time, I repeated the estimation over 10 random

sample splits rather than the 100 used in the original paper. I used parallel computation across 4 cores of my computer to accelerate estimation. The Mean ATE, Median ATE, and their standard errors are computed following the paper’s formulas exactly. Results are written to `output/tables/main_result.tex`, which is included directly in this document via `\input{}`.

4 Replication Results

Table 1: Replication of Table 1, Panel B (Partially Linear Model, 2-fold) from Chernozhukov et al. (2017)

	This Replication	Paper (Ensemble)
Mean ATE (\$)	8970.03	9,043
Std. Error	1406.97	1,432
95% CI	[6212.38, 11727.68]	—
Median ATE (\$)	9017.29	9,061
Median SE	1406.97	1,343

Note: Based on 10 sample splits, 2-fold cross-fitting.

Paper values from Table 1, Panel B, Ensemble column.

Table 2: Replication vs. Paper: Table 1, Panel B (Partially Linear Model, 2-fold)

	RLasso	Reg. Tree	Random Forest	Boosting	Neural Net	Ensemble
<i>This Replication</i>						
Mean ATE (\$)	7603	8583	8757	8770	8792	8970
	(1729)	(1518)	(1426)	(1379)	(1434)	(1407)
<i>Chernozhukov et al. (2017)</i>						
Mean ATE (\$)	7718	8745	9180	8768	9040	9043
	(1796)	(1488)	(1526)	(1451)	(1494)	(1432)

Note: Standard errors in parentheses. Replication based on 10 sample splits, 2-fold cross-fitting.

Paper values from Table 1, Panel B of the online appendix.

This replication closely reproduces the headline result. The Ensemble estimate of the ATE is \$8,970, compared to \$9,043 reported in the paper, a difference of \$73, or less than one percent. The replication standard error of 1,407 is nearly identical to the paper’s 1,432, and the 95% confidence interval [\$6,212, \$11,728] comfortably contains the published point estimate. The Median ATE of \$9,017 is also close to the paper’s median of \$9,061, confirming that the result is not driven by outlier splits.

These small discrepancies are expected for a replication like this and can be easily explained. The original paper averages over 100 random sample splits, but this replication uses only 10. With fewer splits, individual random partitions carry more weight, so the mean estimate is more sensitive to any one particularly favorable or unfavorable draw of the folds. The fact that the two estimates still agree to within one percent despite this reduction suggests

the procedure is very stable. This is consistent with the Neyman orthogonality framework, which suggests that the final treatment effect estimate should not be overly sensitive to small errors in the nuisance estimates or to a particular sample split.

Table 2 shows the method-by-method comparison. The overall pattern is also consistent between the replication and the paper. RLasso produces the lowest ATE in both (\$7,603 vs. \$7,718), reflecting the tendency of ℓ_1 -penalized linear models to underfit when extrapolation is required across folds. Random Forest exhibits the largest difference (\$8,757 vs. \$9,180), which may reflect the greater sensitivity of tree-based methods to the particular sample splits used in estimation. Random Forest and Neural Net sit at the higher end in both cases, and the Ensemble sits close to them, as expected given that it up-weights whichever methods minimize out-of-sample prediction error. Standard errors are broadly comparable across all six methods, with differences generally well below 100. The consistency of these results across very different machine learning approaches can also be seen as a substantive finding, confirming the paper’s central claim that Neyman orthogonality makes the treatment effect estimate robust to the choice of nuisance estimator.

5 What I Learned

Once the helper functions were sourced correctly, reproducing the estimation was fairly straightforward. The hardest part was path management, as the original code assumed all files lived in the same working directory, while the Gentzkow-Shapiro structure separates `input/`, `code/`, and `temp/`. In particular, `Moment Functions.R` internally sources `ML Functions.R` with a bare filename, which required patching to use an absolute path. This was a pure reproducibility issue: code that works perfectly in the author’s environment can break when placed in a different directory structure. This happened to me before I made the fix. If I were the original authors, the one thing that would have made replication easier is absolute rather than relative paths in the helper files, along with a brief README note specifying how the scripts should be run.

More broadly, this replication taught me that Double ML is amazingly stable. Six very different machine learning methods (RLasso, Trees, Random Forest, Boosting, Neural Net, and Ensemble) all produce estimates between approximately \$7,900 and \$9,200, with substantial overlap in their confidence intervals. This stability is exactly what the Neyman orthogonality theory predicts, and seeing it empirically with real-world data was something I could have learned only from doing this replication.

References

- [1] Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., and Newey, W. (2017). “Double/Debiased/Neyman Machine Learning of Treatment Effects.” *American Economic Review: Papers & Proceedings*, 107(5): 261–265.